

Slide 1 — Expressões, Atribuição e Controle

É a apresentação da aula. O tema é entender:

- como os valores são combinados → **expressões**;
- como valores são armazenados → **atribuição**;
- em que ordem o código é executado → **estruturas de controle**.

Slide 2 — Objetivo e ligação com o mundo real

Mostra por que esses assuntos são importantes. Erros de precedência, conversões e estruturas de controle podem fazer um programa executar algo diferente do esperado.

Apresenta três exemplos reais:

- **Apple — “goto fail” (2014)**: um `goto` duplicado fez uma verificação de segurança ser ignorada.
- **Microsoft Zune (2008)**: um `while` entrou em loop infinito.
- **PHP**: comparações com `==` podem fazer conversões inesperadas.

Slide 3 — Roteiro da aula

Mostra a ordem dos assuntos:

1. Quiz inicial.
2. Expressões.
3. Conversões e booleanos.
4. Atribuição.
5. Seleção, laços e `goto`.
6. Exercício para encontrar bugs.

Slide 4 — Quiz de abertura

O objetivo é tentar descobrir a saída dos códigos **antes de executá-los**.

Exemplos importantes:

- Python: `-2 ** 2` → `-4`
- Python: `2 ** 3 ** 2` → `512`
- Java: `1 + 2 + "3" + 4 + 5` → `"3345"`
- JavaScript: `"5" - 2` → `3`, mas `"5" + 2` → `"52"`
- C: `3 > 2 > 1` → `0`
- Python: `3 > 2 > 1` → `True`
- JS: `0 == ""` → `true`

A ideia é mostrar que **cada linguagem possui suas próprias regras**.

Slide 5 — Expressões aritméticas

Explica que uma expressão possui:

- operadores;
- operandos;
- parênteses;
- chamadas de funções.

Apresenta três conceitos principais:

- **precedência:** qual operador vem primeiro;
- **associatividade:** em qual direção operadores de mesma prioridade são avaliados;
- **ordem de avaliação:** quando cada operando é calculado.

Exemplo:

`a + b * c`

Com `a=3`, `b=4`, `c=5`, o resultado é `23`, porque `*` possui precedência maior que `+`.

Também mostra que linguagens diferentes podem escolher regras diferentes, como APL e Lisp.

Slide 6 — Precedência e associatividade na prática

Mostra diferenças entre Python, JavaScript e Java.

Principal exemplo:

`-2 ** 2`

Em Python $\rightarrow -4$, porque `**` vem antes do `-`.

Também mostra:

`2 ** 3 ** 2` $\rightarrow 512$

porque a exponenciação é associativa à direita.

Regra importante: quando estiver em dúvida, use parênteses.

Também compara divisão e resto negativos entre linguagens.

Slide 7 — Ordem dos operandos e efeitos colaterais

Mostra que a ordem em que uma expressão é avaliada pode alterar o resultado quando existem **efeitos colaterais**.

Exemplo:

```
a = a + fun1()
```

Se `fun1()` altera `a`, o resultado pode depender de quando `a` foi lido.

- Java $\rightarrow$ ordem definida da esquerda para a direita.
- C $\rightarrow$ o padrão não define a ordem.
- Go $\rightarrow$ possui regras próprias para avaliação.

A ideia principal é: **funções com efeitos colaterais podem tornar expressões imprevisíveis.**

Slide 8 — Operadores sobrecarregados

Um mesmo operador pode ter diferentes significados.

Exemplo:

+

pode significar:

- soma de números;
- concatenação de textos.

Em Java:

```
'a' + 1  $\rightarrow$  98
```

Mas:

```
" " + 'a' + 1  $\rightarrow$  "a1"
```

Também mostra que algumas linguagens permitem criar sua própria sobrecarga de operadores, como Python, C++, C#, Rust e Kotlin.

Slide 9 — Conversões de tipos

Apresenta:

Alargamento: tipo menor $\rightarrow$ tipo maior.

Exemplo:

```
int  $\rightarrow$  float
```

Estreitamento: tipo maior $\rightarrow$ tipo menor.

Exemplo:

```
double  $\rightarrow$  int
```

Pode ocorrer perda de informação.

Também diferencia:

- **coerção:** conversão automática;
- **cast:** conversão solicitada explicitamente pelo programador.

Go e Rust são mais rígidos e normalmente exigem conversões explícitas.

Slide 10 — Coerção implícita: JavaScript e PHP

Mostra como JavaScript pode converter tipos automaticamente.

Exemplos:

`"5" - 2 → 3`

`"5" + 2 → "52"`

E:

`0 == "" → true`

`0 == "0" → true`

`0 === "0" → false`

Por isso, a recomendação apresentada é usar:

`===` e `!==`

quando quiser evitar conversões automáticas.

Também apresenta o problema de **type juggling** do PHP.

Slide 11 — Expressões relacionais e booleanas

Mostra como linguagens tratam valores verdadeiros e falsos.

Em C:

`3 > 2 > 1`

não significa o que parece.

Primeiro:

`3 > 2 → 1`

Depois:

`1 > 1 → 0`

Já Python entende:

`3 > 2 > 1`

como uma comparação encadeada e retorna `True`.

Também explica **truthiness**:

- `0 → falso`;
- `"" → falso`;
- `[] → falso em Python`;
- `"0" → verdadeiro`.

Java, Go e Rust exigem um valor booleano no `if`.

Slide 12 — Avaliação em curto-circuito

Explica que nem todos os operandos precisam ser avaliados.

Com:

`&&`

se a primeira condição já for falsa, a segunda pode nem ser executada.

Com:

`||`

se a primeira for verdadeira, a segunda pode não ser executada.

Isso evita erros, como acessar algo inexistente.

Também mostra:

`qtd || 10`

pode transformar `0` em `10`.

Já:

`qtd ?? 10`

só substitui quando o valor é `null` ou `undefined`.

Slide 13 — Sentenças de atribuição

Explica formas de atribuir valores:

Atribuição simples:

```
x = 10
```

Atribuição composta:

```
x += 5
```

equivale a:

```
x = x + 5
```

Também apresenta atribuições múltiplas em linguagens como Python.

Um ponto importante é diferenciar:

= → atribuição

== → comparação

E algumas linguagens permitem atribuições dentro de expressões, enquanto outras evitam isso.

Slide 14 — Atribuição de modo misto

Trata da atribuição entre tipos diferentes.

Exemplo:

```
int → double
```

pode ser permitido automaticamente em algumas linguagens.

O slide destaca que conversões automáticas podem esconder erros.

Também mostra uma pegadinha:

```
double taxa = acertos / total * 100;
```

Se `acertos` e `total` forem inteiros, a divisão acontece primeiro como divisão inteira.

Com `7 / 10`, temos:

```
0
```

e depois esse `0` vira `double`.

Resultado:

```
0.0%
```

Slide 15 — Estruturas de controle

Apresenta as três estruturas fundamentais:

1. **Sequência** → comandos executados em ordem.
2. **Seleção** → escolher um caminho (`if`, `switch`).
3. **Iteração** → repetir comandos (`while`, `for`).

O slide cita Böhm e Jacopini: essas três estruturas são suficientes para representar qualquer algoritmo estruturado.

Slide 16 — Seleção de dois caminhos: “else pendente”

Mostra o problema de `if` dentro de `if`.

Exemplo:

```
if (sum == 0)
    if (count == 0)
        result = 0;
    else
        result = 1;
```

O `else` pertence ao **if mais próximo que ainda não possui `else`**.

A indentação visual não muda essa regra em C, Java e C++.

Por isso, usar **chaves `{ }`** evita ambiguidades.

Slide 17 — Seleção múltipla: `switch` e `match`

Compara diferentes linguagens.

Em C:

```
switch
```

pode ter **fall-through**, ou seja, continuar executando os próximos `case` se não houver `break`.

Java moderno possui:

```
case X -> ...
```

que evita esse problema.

Rust possui:

```
match
```

e exige que os casos sejam tratados de forma **exaustiva**.

Go também possui `switch`, mas cada caso termina automaticamente, a menos que seja usado `fallthrough`.

Slide 18 — Laços controlados por contador

Explica o `for`.

Em C:

```
for (int i = 0; i < 10; i++)
```

A variável pode ser modificada dentro do laço.

Em Python:

```
for i in range(10):
```

o `for` percorre os valores fornecidos pelo `range`.

Então alterar `i` dentro do corpo não funciona da mesma forma que em C.

Slide 19 — Laços lógicos, saídas e iteradores

Apresenta:

- `while`;
- `do-while`;
- `break`;
- `continue`;
- `iteradores`;
- `for...in`;
- `for...of`.

Uma pegadinha muito importante do JavaScript:

```
for (const i in notas)
```

percorre os **índices/chaves**.

Enquanto:

```
for (const v of notas)
```

percorre os **valores**.

Também apresenta o caso do **Zune**, em que um `while` ficou infinito porque a variável de controle não era alterada em determinada situação.

Slide 20 — Desvio incondicional: `goto`

Explica o comando:

```
goto
```

Ele permite pular diretamente para um determinado ponto do código.

O problema é que muitos `goto` podem deixar o programa difícil de entender e manter.

Apresenta o famoso caso “**goto fail**” da Apple, em que um `goto` duplicado fez com que uma verificação de segurança fosse ignorada.

Também cita Dijkstra e sua crítica ao uso indiscriminado de `goto`.

Slide 21 — Comandos protegidos de Dijkstra

Apresenta uma ideia de Dijkstra para seleção e repetição.

Várias condições podem ser verdadeiras ao mesmo tempo e o sistema pode escolher uma delas.

A ideia é trabalhar com **guardas**, por exemplo:

```
if x >= y -> ...  
[] y >= x -> ...  
fi
```

Quando `x == y`, as duas condições são verdadeiras e qualquer uma pode ser escolhida.

O conceito influenciou sistemas como o `select` da linguagem Go.

Slide 22 — Exercício: encontre o bug 1/3

Dois exercícios.

JavaScript:

O código usa `for...in` em um array e acaba somando os índices em vez dos valores.

Resultado:

```
"0012"
```

Correção:

```
for (const p of precos)
```

Python:

```
d = d or 10
```

faz com que 0 seja considerado falso.

Assim, um desconto de 0% acaba virando 10%.

Slide 23 — Exercício: encontre o bug 2/3

Java:

```
switch sem break.
```

O programa continua passando pelos próximos `case` até chegar ao `default`.

Resultado:

```
inválido
```

C:

```
if (saque > saldo);
```

O ; encerra o `if`.

Então o `printf` seguinte sempre é executado.

Slide 24 — Exercício: encontre o bug 3/3

Go:

```
x := x * 2
```

O `:=` cria uma **nova variável** dentro do bloco.

Por isso:

- dentro $\rightarrow 20$
- fora $\rightarrow 10$

A correção é:

```
x = x * 2
```

Java:

```
double taxa = acertos / total * 100;
```

Resultado:

```
0.0%
```

porque `7 / 10` é calculado como divisão inteira antes de virar `double`.

Slide 25 — Síntese

É o resumo das principais armadilhas:

- **Precedência** → use parênteses.
- **Efeitos colaterais** → podem mudar o resultado.
- **Conversões** → podem esconder erros.
- **Booleanos** → linguagens possuem regras diferentes.
- **Atribuição** → cuidado com `=` e `==`.
- **Switch** → cuidado com `break`/fall-through.
- **Laços** → cuidado com loops sem progresso.
- **Goto** → pode prejudicar a legibilidade.

A ideia principal é que linguagens mais modernas tentam **impedir determinadas classes de bugs**.

Slide 26 — Atividades complementares

É o último slide e indica assuntos para estudar por conta própria:

- expressões em Ruby;
- expressões em Lisp;
- expressões condicionais;
- implementação de `switch`;
- laços em linguagens funcionais;
- iteradores e blocos de Ruby;
- texto de Dijkstra sobre `goto`.